

Tecnicatura Universitaria en Programación

- **Asignatura:** Programación II
- **Alumno:** Santino Nicolas Boscatto
- **Fecha de entrega:** 22/06/2026
- **Trabajo Practico Integrador:** Food Store – Sistema de Gestión de Pedidos de Comida

1. Carátula

2. Índice

3. Marco Teórico y Conceptual

- 3.1 Programación Orientada a Objetos
- 3.2 Colecciones y Programación Funcional
- 3.3 Excepciones Personalizadas
- 3.4 Genéricos y Reutilización de Código
- 3.5 Arquitectura de Software en Capas

4. Decisiones Técnicas y Arquitectura del Sistema

- 4.1 Organización del Proyecto
- 4.2 Modelo de Dominio
- 4.3 Servicios y Reutilización de Código
- 4.4 Interfaz de Usuario
- 4.5 Utilidades y Datos de Prueba
- 4.6 Baja Lógica y Creación de Pedidos

5. Pruebas y Validaciones

6. Desafíos y Soluciones de Diseño

- 6.1 Gestión de IDs Únicos
- 6.2 Reutilización de Lógica CRUD
- 6.3 Creación de Pedidos y Consistencia de Datos
- 6.4 Manejo de Errores e Interacción con el Usuario

7. Conclusiones

8. Bibliografía

3. Marco Teórico y Conceptual

3.1 Programación Orientada a Objetos

El sistema Food Store se fundamenta en los cuatro pilares de la Programación Orientada a Objetos (POO), aplicados de manera transversal en todo el proyecto.

Herencia: Todas las entidades del dominio (Categoria, Producto, Usuario, Pedido, DetallePedido) heredan de una clase abstracta Base. Esta clase encapsula los atributos comunes a todo el sistema: id (identificador único autogenerated), eliminado (marca para baja lógica) y createdAt (fecha y hora de creación). La herencia evita la duplicación de código, facilita el mantenimiento y establece un contrato común: todas las entidades deben implementar toString() y cuentan con los métodos definidos en Base.

Encapsulamiento: Cada atributo de las entidades es privado y se accede mediante getters y setters públicos. Los setters incorporan validaciones que garantizan la integridad de los datos antes de que se modifiquen. Por ejemplo, el setter de precio en Producto rechaza valores negativos, y el de mail en Usuario exige un formato válido.

Polimorfismo: Se utiliza en distintos puntos del sistema. Por un lado, todas las entidades heredan de la clase Base, que define métodos comunes como equals() y toString(). Cada entidad implementa su propia versión de toString(), permitiendo que una misma llamada produzca una representación diferente según el objeto que se esté mostrando. Por otro lado, la clase genérica CrudGenerico concentra las operaciones CRUD comunes y delega en métodos que cada servicio implementa de acuerdo con las necesidades de su entidad. De esta manera, una misma estructura puede trabajar con categorías, productos, usuarios o pedidos, adaptando su comportamiento según el tipo concreto con el que se esté operando. Además, la interfaz Calculable define el método calcularTotal(), que es implementado por Pedido para calcular el importe total a partir de sus detalles.

Abstracción: El sistema modela conceptos del dominio de un negocio de comidas mediante clases como Usuario, Producto, Categoria y Pedido, representando únicamente los atributos y comportamientos relevantes para el problema. Además, se utiliza la clase abstracta Base para concentrar características comunes a todas las entidades y la interfaz Calculable para definir comportamientos generales sin depender de una implementación concreta.

3.2 Colecciones y Programación Funcional

Para las operaciones de búsqueda y filtrado se utiliza la API de Streams de Java junto con expresiones lambda. Por ejemplo, el método listarTodos() de CrudGenerico emplea un stream para devolver únicamente los registros activos. Se optó por este enfoque porque permite escribir el código de forma más clara y concisa que utilizando bucles for tradicionales con múltiples condiciones, facilitando además su lectura y mantenimiento.

3.3 Excepciones Personalizadas

El sistema define seis excepciones propias, todas herederas de RuntimeException: TextoInvalidoException, NumeroNegativoException, NumeroNoPositivoException, ValidacionException, EntidadNoEncontradaException y MailDuplicadoException.

Estas excepciones no agregan un comportamiento propio, sino que sirven para representar errores específicos del sistema de una manera más clara. Esto mejora la legibilidad del código y facilita el manejo de errores, ya que permite distinguir rápidamente qué problema ocurrió y actuar en base a ello.

3.4 Genéricos y Reutilización de Código

Con el objetivo de reducir la duplicación de código, se implementó una base genérica para los servicios mediante la interfaz `CrudService<T>` y la clase abstracta `CrudGenerico<T extends Base>`. Esto permite reutilizar las operaciones CRUD comunes para todas las entidades del sistema. De esta manera, funcionalidades como agregar, buscar, listar o eliminar lógicamente se programan una sola vez, mientras que cada servicio incorpora únicamente las validaciones y reglas de negocio que le corresponden.

3.5 Arquitectura de Software en Capas

El proyecto se organizó en capas para separar responsabilidades y evitar mezclar la lógica del sistema con la interacción por consola. La capa **UI** gestiona los menús y la comunicación con el usuario, la capa **Services** implementa las reglas de negocio y las operaciones CRUD, y la capa **Entities** representa los objetos del dominio. También se incluyen paquetes de **Utilidades** y **Excepciones** para centralizar funciones comunes y el manejo de errores. Esta organización ayuda a mantener el código más ordenado y facilita futuras modificaciones.

4. Decisiones Técnicas y Arquitectura del Sistema

4.1 Organización del proyecto

El proyecto se encuentra dividido en paquetes según su responsabilidad:

- **entities:** contiene las entidades del dominio (Categoria, Producto, Usuario, Pedido, DetallePedido).
- **services:** implementa la lógica de negocio y las operaciones CRUD. Dentro de este paquete se encuentra la implementación genérica del CRUD, compuesta por la interfaz `CrudService<T>` y la clase abstracta `CrudGenerico<T>`.
- **ui:** administra la interacción con el usuario mediante menús por consola.
- **exceptions:** agrupa las excepciones personalizadas utilizadas para validaciones y manejo de errores.
- **interfaces:** define contratos reutilizables entre distintas clases.
- **utils:** contiene métodos auxiliares validadores y la carga inicial de datos de prueba.
- **enums:** reúne los tipos enumerados utilizados por el sistema.

Esta organización permite mantener el código más ordenado y facilita la localización de responsabilidades dentro de la aplicación.

4.2 Modelo de dominio

Las entidades representan los conceptos principales del negocio: categorías, productos, usuarios, pedidos y detalles de pedido.

La clase abstracta Base concentra características comunes a todas las entidades, como el identificador único, la fecha de creación, la baja lógica y la implementación de equals() y hashCode() basados en el ID.

Las relaciones del sistema son:

Categoría => Producto: relación uno a muchos (1:N). Una categoría puede contener varios productos, mientras que cada producto pertenece a una única categoría.

Usuario => Pedido: relación uno a muchos (1:N). Un usuario puede realizar varios pedidos, pero cada pedido pertenece a un único usuario.

Pedido => DetallePedido: relación uno a muchos (1:N). Un pedido puede contener varios detalles, mientras que cada detalle pertenece a un único pedido.

Producto => DetallePedido: relación uno a muchos (1:N). Un producto puede aparecer en varios detalles de pedido, pero cada detalle referencia a un único producto.

Pedido <=> Producto: relación muchos a muchos (N:M) implementada mediante la entidad intermedia DetallePedido. Un pedido puede contener varios productos y un producto puede formar parte de múltiples pedidos.

Las validaciones básicas se realizan dentro de las propias entidades mediante métodos auxiliares de la clase Utilitario, evitando que se puedan crear o modificar objetos con datos inválidos.

4.3 Servicios y reutilización de código

La lógica de negocio se concentra en la capa services.

Para evitar duplicar código se implementó una solución genérica basada en la interfaz CrudService<T> y la clase abstracta CrudGenerico<T extends Base>, que proporciona funcionalidades comunes como:

- Crear entidades.
- Buscar por ID.
- Listar registros activos.
- Actualizar información.
- Realizar bajas lógicas.

Cada servicio concreto (ServicioCategoria, ServicioProducto, ServicioUsuario y ServicioPedido) reutiliza esta funcionalidad y agrega las reglas específicas de su entidad.

Por ejemplo:

ServicioCategoria valida nombres únicos e impide eliminar categorías con productos activos.

ServicioUsuario valida el formato y la unicidad del correo electrónico.

ServicioPedido controla la creación de pedidos y las transiciones permitidas entre estados.

4.4 Interfaz de usuario

La interacción con el usuario se realiza mediante menús por consola organizados en clases independientes.

Cada menú se encarga de solicitar datos, mostrar resultados y comunicarse con la capa de servicios para ejecutar las distintas operaciones. Además, captura las excepciones generadas tanto por los servicios como por los métodos de validación de Utilitario, mostrando mensajes claros al usuario sin interrumpir la ejecución del programa.

También se incorporaron validaciones de entrada para reducir errores durante la carga de datos y mejorar la experiencia de uso. De esta forma, el sistema puede detectar y comunicar problemas de manera temprana antes de que afecten a la lógica de negocio.

4.5 Utilidades y datos de prueba

La clase Utilitario centraliza validaciones reutilizables, como comprobaciones de texto, números y formato de correo electrónico.

Por su parte, DataSeed genera datos iniciales de prueba que permiten verificar rápidamente todas las funcionalidades del sistema sin necesidad de cargar información manualmente.

4.6 Baja lógica y creación de pedidos

Todas las eliminaciones se realizan mediante baja lógica, marcando los registros como eliminados en lugar de removerlos físicamente de las colecciones. Esto permite conservar información histórica y mantener la consistencia de los datos.

En el caso de los pedidos, la creación se realiza de forma controlada para evitar inconsistencias. Si ocurre un error durante la carga de alguno de sus detalles, el pedido no se registra y la operación se cancela por completo.

5. Pruebas y validaciones

Se realizaron pruebas funcionales sobre las cuatro áreas principales del sistema (categorías, productos, usuarios y pedidos), verificando tanto los casos de éxito como los casos de error.

Entre las validaciones comprobadas se encuentran:

- Creación, edición, consulta y eliminación lógica de todas las entidades.
- Validación de unicidad de nombres de categoría y correos electrónicos.
- Restricciones para eliminar categorías con productos asociados.
- Validación de precios, stock y cantidades inválidas.
- Creación de pedidos con múltiples detalles y cálculo automático del total.
- Validación de transiciones de estado de los pedidos.
- Conservación del historial mediante bajas lógicas.
- Comportamiento atómico en la creación de pedidos ante errores en los detalles.

Además, se verificó que todas las excepciones sean capturadas correctamente por la interfaz de usuario, mostrando mensajes descriptivos sin cortar la ejecución de la aplicación.

Ejemplo 1. Intento de crear una categoría con un nombre ya existente. El sistema detecta la duplicidad y muestra un mensaje de error sin interrumpir la ejecución.

```
=== SISTEMA DE PEDIDOS (FOOD STORE) ===
1. Categorías
2. Productos
3. Usuarios
4. Pedidos
0. Salir
Seleccione: 1

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione: 2
Nombre: Bebidas
Descripción: Gaseosas, Jugos
Error, Ya existe una categoría con el nombre Bebidas

--- CATEGORIAS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
0. Volver
Seleccione:
```

Ejemplo 2. Intento de cambiar el estado de un pedido desde **TERMINADO** a **PENDIENTE**. La validación de reglas de negocio impide la operación y notifica el error correspondiente.

```
--- PEDIDOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
5. Ver detalles
0. Volver
Seleccione: 3
Listado de pedidos:
Pedido #16 | Usuario: Enzo Vitale (12) | Fecha: 2026-06-10 | Estado: PENDIENTE | FormaPago: TARJETA | Total: $13500,00
Pedido #19 | Usuario: Jeremias Brochero (13) | Fecha: 2026-06-11 | Estado: CONFIRMADO | FormaPago: EFECTIVO | Total: $10100,00
Pedido #22 | Usuario: Enzo Vitale (12) | Fecha: 2026-06-09 | Estado: TERMINADO | FormaPago: TRANSFERENCIA | Total: $12500,00
Pedido #24 | Usuario: Santino Boscatto (11) | Fecha: 2026-06-08 | Estado: CANCELADO | FormaPago: TARJETA | Total: $4000,00
Introduzca el Id del pedido a editar: 22
Pedido #22 | Usuario: Enzo Vitale (12) | Fecha: 2026-06-09 | Estado: TERMINADO | FormaPago: TRANSFERENCIA | Total: $12500,00
Nuevo estado:
1. PENDIENTE
2. CONFIRMADO
3. TERMINADO
4. CANCELADO
Seleccione (deje vacío para mantener el estado 'TERMINADO'): 1
Error, Un pedido TERMINADO no puede cambiar de estado

--- PEDIDOS ---
1. Listar
2. Crear
3. Editar
4. Eliminar
5. Ver detalles
0. Volver
Seleccione: |
```

6. Desafíos y soluciones de diseño

Gestión de IDs únicos

Uno de los aspectos que tuve que resolver fue la asignación de IDs únicos para todas las entidades. Decidí que los identificadores se generaran únicamente al momento de crear una entidad de forma exitosa. Por este motivo, la asignación del ID se realiza dentro de `CrudGenerico`, después de que la entidad supera todas las validaciones. De esta manera, los intentos fallidos de creación no consumen identificadores y la numeración se mantiene ordenada.

Reutilización de lógica CRUD

Como el sistema trabaja con categorías, productos, usuarios y pedidos, implementar un CRUD completo para cada entidad podía generar una gran cantidad de código repetido. Para evitarlo, diseñé una solución genérica basada en la interfaz `CrudService<T>` y la clase abstracta `CrudGenerico<T extends Base>`. Allí se concentra la lógica común de búsqueda, listado y eliminación lógica, mientras que cada servicio concreto se encarga únicamente de sus validaciones y reglas específicas. Esto permitió reducir la duplicación de código y facilitar el mantenimiento del proyecto.

Creación de pedidos y consistencia de datos

Los pedidos son la entidad más compleja del sistema porque involucran usuarios, productos y detalles de pedido. Un desafío importante fue evitar que se generaran pedidos incompletos cuando ocurría un error durante la carga de los detalles. Para resolverlo, se implementó un método de creación que valida previamente toda la información necesaria y, ante cualquier error, cancela la operación completa. De esta forma, el sistema evita almacenar pedidos inconsistentes o parcialmente cargados.

Manejo de errores e interacción con el usuario

Otro desafío importante fue mejorar el flujo de interacción para evitar que el usuario realizara pasos innecesarios antes de descubrir un error. Por ejemplo, se incorporaron validaciones tempranas para detectar campos vacíos antes de solicitar el resto de los datos, y controles para impedir operaciones como editar o eliminar cuando no existen registros disponibles. También se verificó la existencia de una entidad antes de pedir confirmaciones de eliminación. Además, los menús capturan las excepciones generadas por los servicios y por las validaciones de Utilitario, mostrando mensajes claros sin interrumpir la ejecución de la aplicación. Esto permitió obtener una experiencia de uso más fluida y consistente.

7. Conclusiones

El desarrollo de Food Store me permitió aplicar de forma práctica los principales conceptos trabajados en Programación II, especialmente Programación Orientada a Objetos, colecciones, excepciones personalizadas y genéricos.

Uno de los aspectos más importantes del proyecto fue la reutilización de código mediante la implementación de un CRUD genérico, lo que permitió evitar duplicaciones y mantener una estructura más simple y fácil de extender. Del mismo modo, la separación en capas ayudó a

organizar las responsabilidades del sistema, diferenciando claramente la lógica de negocio, las entidades del dominio y la interacción con el usuario.

Además de cumplir con los requisitos funcionales, el desarrollo me permitió enfrentar desafíos relacionados con validaciones, manejo de errores y consistencia de datos, fortaleciendo mi comprensión sobre diseño y mantenimiento de aplicaciones en Java. Como resultado, se obtuvo una aplicación funcional, organizada y alineada con los objetivos propuestos para el trabajo práctico integrador.

8. Bibliografía y Herramientas Utilizadas

- Oracle. (2025). The Java Tutorials. <https://docs.oracle.com/javase/tutorial/>
- Oracle. Java Platform, Standard Edition 21 Documentation. <https://docs.oracle.com/en/java/javase/21/>
- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). Design Patterns: Elements of Reusable Object-Oriented Software. Addison-Wesley.
- Bloch, J. (2018). Effective Java (3rd ed.). Addison-Wesley.
- Apache Software Foundation. (2025). Apache Maven Project. <https://maven.apache.org/>